

Machine Vision and Sparsity

Candidate nr. 19

Abstract—This paper addresses the challenge of optimizing image classification for PillCam endoscopic procedures to enhance diagnostic efficiency. We implemented two deep learning objectives: first, classifying images as 'Wall' or 'Rest' to filter out non-diagnostic images. Second, classifying images into four quality classes despite noisy labels from manual classifications. Utilizing sparsified ResNet models, we achieved high classification accuracy while reducing model complexity. The results demonstrate that controlling image capture based on real-time classification can help diagnostic yield and minimize irrelevant data.

Index Terms—Deep Learning, Sparsity, ResNet, PillCam

I. INTRODUCTION

Two supervised learning objectives were implemented and studied. First, and the easier objective, is regarding a dataset of images conducted by the PillCam, where task is to classify the images as Wall or Rest. This refers to images that are taken by the pill while the pill is positioned with the cameras looking at the walls inside of the patient digestive system. We do not want images of walls as it does not help in diagnosing the patients, the Rest class images are images as the pill has its cameras positioned in other angles other than Walls.

In the second task, 500 of the same type images were provided by the PillCam, but this time these images were supposed to be classified into 4 classes of quality. These images are classified by medical professionals manually, meaning that the labeling contain some noise.

Given the data were supposed to find models which can perform well for the tasks at hand, and manage to sparsify these models. Potentially improving the performance of the PillCam by controlling which images are taken, and which are not. By doing this we maximize the amount of images that can be used in diagnosing the patients, and minimizing the amount of images which are not of use for that same process.

II. THEORY

A. Sparsity in Deep Learning

Sparsity in Deep Learning has gained significant attention as researchers and engineers seek more efficient ways to train and deploy deep networks. This is needed due to increase in computational and memory demands of larger models, sparsity offers a way to reduce these costs by utilizing only a subpart of parameters within a network. The idea behind sparsification is to remove less significant weights or activations in the network, resulting in a sparse model that performs similarly to,

or sometimes even better than, the original dense model, but with significantly less parameters. This reduction in size and complexity not only decreases storage requirements but also speeds up inference through lessening the number of FLOPs. FLOPs represent the total number of arithmetic operations, like additions and multiplications, that a neural network performs during a forward or backward pass. This makes it possible to deploy advanced models on constrained devices, such as embedded systems or mobile applications.

B. Model

1) *ResNet*: The core concept behind ResNet architectures is the use of residual connections, which allow gradients to flow more easily through deeper layers by skipping certain blocks in the network. This residual learning framework lessens the vanishing gradient problem that typically arises in deep networks, allowing the model to achieve higher accuracy without adding unnecessary complexity.

ResNet architectures come in various depths, with ResNet18 being one of the shallower versions comprising 18 convolutional layers. Despite its relative simplicity, ResNet18 retains the benefits of residual learning while being computationally efficient, making it suitable for applications with limited resources or when faster training times are desired.

2) *Optimizer*: Optimizers play an important role in training neural networks by updating the weights to minimize the loss function. They decide how the model's parameters are changed based on the gradients computed during backpropagation.

One simple but widely used optimizer is the *Stochastic Gradient Descent*, it updates parameters using the gradient of the loss function with respect to the weights. However, SGD with a fixed learning rate can be inefficient and may converge slowly, especially in the presence of noisy gradients or sparse data. It might also struggle with local minimums when converging.

To overcome these limitations, adaptive learning rate methods like *Adam*, in full, Adaptive Moment Estimation, have been developed. It combines the advantages of two other extensions of SGD: *Adaptive Gradient Algorithm* (AdaGrad) and *Root Mean Square Propagation* (RMSProp). It keeps per-parameter learning rates that are adapted based on the first and second moments of the gradients.

```

Require:  $\alpha$ : Stepsize
Require:  $\beta_1, \beta_2 \in [0, 1]$ : Exponential decay rates for the moment estimates
Require:  $f(\theta)$ : Stochastic objective function with parameters  $\theta$ 
Require:  $\theta_0$ : Initial parameter vector
 $m_0 \leftarrow 0$  (Initialize 1st moment vector)
 $v_0 \leftarrow 0$  (Initialize 2nd moment vector)
 $t \leftarrow 0$  (Initialize timestep)
while  $\theta_t$  not converged do
   $t \leftarrow t + 1$ 
   $g_t \leftarrow \nabla_{\theta} f_t(\theta_{t-1})$  (Get gradients w.r.t. stochastic objective at timestep  $t$ )
   $\hat{m}_t \leftarrow \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$  (Update biased first moment estimate)
   $\hat{v}_t \leftarrow \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$  (Update biased second raw moment estimate)
   $\tilde{m}_t \leftarrow \hat{m}_t / (1 - \beta_1^t)$  (Compute bias-corrected first moment estimate)
   $\tilde{v}_t \leftarrow \hat{v}_t / (1 - \beta_2^t)$  (Compute bias-corrected second raw moment estimate)
   $\theta_t \leftarrow \theta_{t-1} - \alpha \cdot \tilde{m}_t / (\sqrt{\tilde{v}_t} + \epsilon)$  (Update parameters)
end while

```

Fig. 1: Pseudo-Code of the ADAM optimization process. From the paper [1]

3) *Loss Function*: The loss function, also known as the cost function, quantifies the difference between the predicted output of the network and the actual target output. It serves as a guide for the optimizer in adjusting the weights in the network.

For classification tasks with multiple classes, the *Cross-Entropy Loss* is commonly used. If the classification is done only based on two classes, the Cross-Entropy Function becomes Binary Cross-Entropy. The binary function is simply derived from the full Cross-Entropy function. See the Cross-Entropy Loss function below:

$$CE = - \sum_i^N \sum_i^C y_i \log(z_i) \quad (1)$$

here, y denotes true label while z is the prediction of the model as a stochastic probability vector, C is the number of Classes and N denotes the number of Samples. If we assume two class problem, then the equation for CE becomes Binary, and looks as follow.

$$BCE = - \sum_{i=1}^N [y_i \log(z_i) + (1 - y_i) \log(1 - z_i)] \quad (2)$$

III. METHODOLOGY

A. Pruning

The Pruning it self, which is the action of sparcification, was done using unstructured l1 pruning method. This method simply Prune the tensors by removing the units which hold the lowest L1-norm values, meaning it removes the tensors which are least active, L1-norm helps when working with values with many zeros also referred to as sparse data. The equation of L1-norm is as follows:

$$\|v\|_1 = \sum_i |v_i| \quad (3)$$

The number of tensors pruned were a set percentage. The process of pruning which was used in this paper is inspired by the SWAT(Sparse Weight Activation Training) paper [2].

In our case Pruning the weights in forward pass and backward pass was done. For each epoch the model would be pruned for a certain percentage which increased for each epoch. Meaning the model was gradually pruned to be smaller and smaller from 0% pruned to lets say 50% pruned over 20 epochs. In the code it was implemented as follow:

```

current_sparsity = initial_sparsity +
    epoch * sparsity_step

```

here, the `current_sparsity` denotes the percentage of the tensors to be pruned for the current epoch, `initial_sparsity` is the percentage of sparsification to begin with on the first epoch for more aggressive sparcification, then the `sparsity_step` is a value which increases the current sparsification of the epoch by a value which is calculated as $final_sparsity/epochs$, which simply produces a homogeneous sparsification increase from the initial sparsity up to the desired sparsity levels. See figure below as it shows how the sparsification can look like in training.

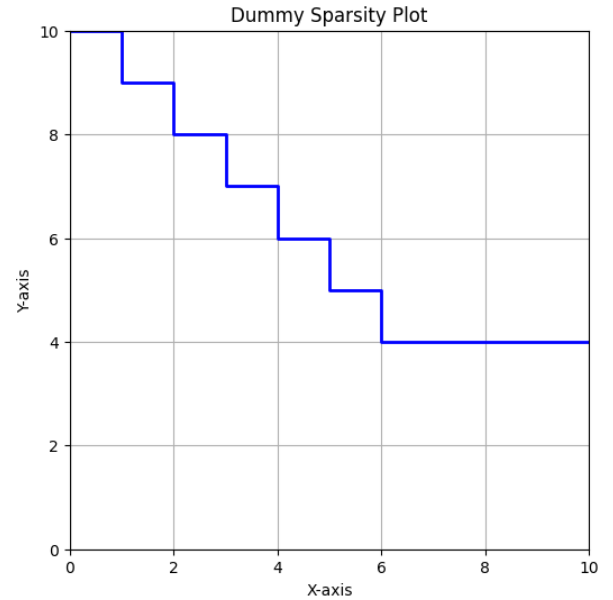


Fig. 2: A simple dummy staircase plot, visualizing how the sparsification is applied in training, where initial sparsity is 0%, and is increased gradually to 60% pruned tensors. Thus, leaving 40% of the parameters.

After pruning process was completed, the model was fine-tuned by training retraining it on the same data one more without any pruning involved. This was done for some small number of epochs.

B. Data

For the experiments, a dataset composed of images taken by the PillCam™ COLON 2 capsule was used. There were two tasks at hand, and for the second one data augmentation techniques were applied. This was done mainly due to lack

of images to train the model on. The augmentation of data included rotations of 90 and 45 degrees applied to the images, which helped simulate various perspectives and reduce overfitting. This could be done due to an important aspect of CNNs, which is spacial invariance. Spacial invariance means that an object in an image can be placed anywhere on the image and rotated, and still be viewed as that same object. These transformations provide the model with slightly changed versions of the same data, helping it to generalize unseen data by making it less sensitive to orientation variations.

1) *First Task (Wall vs Rest)*: In this task no augmentation was applied, the data was simply loaded on to DataLoader using pytorch. Below figures show what kind of images we wish to keep and discard. The images labeled as Rest show more of the colon. These images can then be used to find for example polyps in some part of the digestive system. The Wall class images show us nothing of importance which can be applied in diagnostics.



Fig. 3: Two images with label Rest. These images are of type which we wish to keep for diagnostics.



Fig. 4: Two images with label Wall. These images are of type which we wish to reduce, since these images hold no value.

2) *Second Task (Quality)*: In the task classifying 500 images to four quality classes some data augmentation was necessary to reach reasonable testset accuracy such that pruning was possible to implement without extreme performance loss. The images were labeled by hand by some scandinavian medical professionals, and the classes are 0,1,2 and 3. Where 0 is Poor quality, 1 is Ok quality, 2 is Good, and 3 is Great/Best

quality. These images have some human noise in labeling. Figure below shows how an image looks like after it is rotated.



Fig. 5: Visualization of one of the images from original dataset which shows how the images were augmented by rotation with 45 deg and 90 deg

After applying rotations the dataset of 500 images became 4000, where 3200 of these were used for training and validation, and the rest, which was 800 images, was used for testing. In the discussion part of the paper four more images are displayed in figures 9 to 12, showcasing image labels.

C. Models

ResNet18 was selected as the model architecture due to its balance of performance and efficiency, making it good for the tasks. ResNet18 has a strong baseline performance across various image classification tasks. It was used as the main model in both of the tasks. Although, in quality classification task a bigger model was previewed which was ResNet50. The reason for this choice of model is the idea of increasing the accuracy on the limited dataset by increasing the size of the model, and prune it to a similar size or even smaller than the ResNet18 while at the same time maintaining the performance. The size differences of the models are quite significant.

ResNet18 was initialized with default weights in both the tasks, providing a robust starting point without requiring extensive pretraining. Its relatively low parameter count, compared to larger ResNet versions, also contributes to faster training times and reduced computational costs, matching our goal of a sparsified model.

D. Cross-Validation

Cross-validation is method used to evaluate and improve model performance by splitting the dataset into multiple

subsets, or "folds." In K-fold cross-validation, the data is split into k parts, the model is trained on $k - 1$ of these folds and validated on the remaining one. This process is repeated k times, each time with a different fold as the validation set.

In deep learning, cross-validation is particularly useful for tasks with limited data, it provides a robust evaluation of model performance across various subsets of the dataset. All of the training in quality task was done using 5fold, meaning that the trainingset was split into 5 subsets.

IV. RESULTS

A. Wall or Rest Task

Wall or Rest task where binary classification was done to classify if the image provided by the PillCam is a Wall or Not a Wall. See below how the trained models with different sparsity levels perform on the test data.

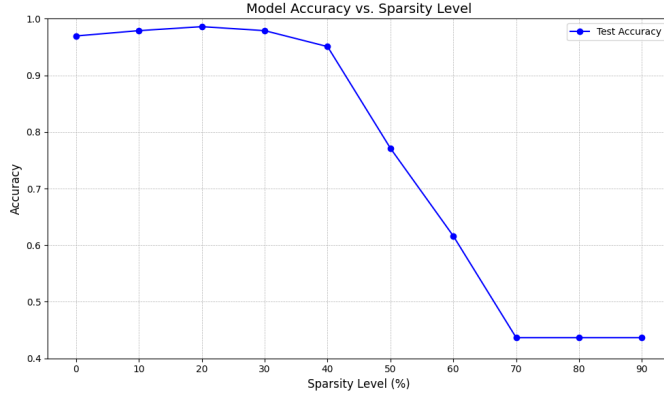


Fig. 6: Different ResNet18 models, all sparsified from 0% to and increased sparsity level. The points are the models accuracy on the testset after training, pruning and finetuning. The first model is pruned from 0% to 0%, second from 0% to 10%, and so on, up to the last model which was gradually pruned from 0% up to 90%.

In the figure above its easily observed that each model drops in performance after pruning levels reach 30% and up. Meaning that the model loses too many parameters and thus can not learn enough patterns from the given data, no matter how much data there is.

B. Quality Task

Comparison between the ResNet18 and ResNet50 with size trade offs but bigger model can be pruned more.

Model	Data Augmentation	Mean Accuracy (%)	Variance
ResNet18	Original Data	70.40	8.64
ResNet50	Original Data	74.80	16.16
ResNet18	90-degree Rotations	81.05	4.86
ResNet50	90-degree Rotations	94.18	1.23
ResNet18	90 and 45-degree Rotations	89.00	0.38
ResNet50	90 and 45-degree Rotations	95.25	1.18

TABLE I: Mean accuracies and variances of ResNet18 and ResNet50 under different data augmentation settings.

The table shows us how the accuracy is increased based on the different size datasets. Mean and Accuracy are calculated based on all the Folds in cross-validation.

1) *Pruning of ResNet*: Below can be seen how pruning affected the accuracy of ResNet18 and ResNet50.

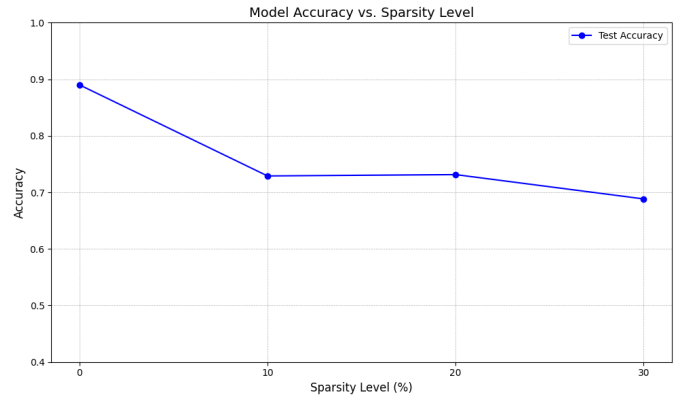


Fig. 7: Increasingly pruned ResNet18 model on the quality task.

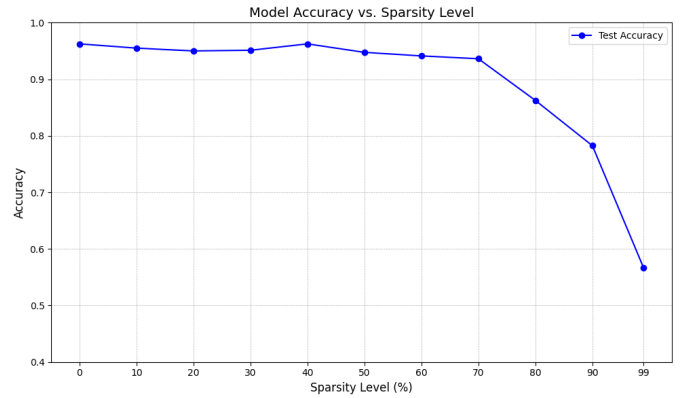


Fig. 8: Increasingly pruned ResNet50 model on the quality task.

Above figures show how the different models react to pruning of increasing levels. 70% is where the model will be benchmarked at, as too poor at classifying the images.

V. DISCUSSION

A. Wall or Rest Task

Since enough data was provided and it was clean and well labeled, the task was easily digested by a simple ResNet18 model. The features which the models had to learn are easily discriminated with fewer parameters. For this simple binary classification problem ResNet18 model performs more than good enough. Also, as seen in figure 6, the model can also be pruned quite significantly. This fact makes the ResNet18 a great performer for this task, with above 90% accuracy after being 40% pruned of its weights.

B. Quality Task

As it could be seen in the Table 1, we observe that adding more data makes the models much more robust as the mean accuracy on the testset increase, but the variance can also be observed to decrease meaning the model becomes quite stable as well. As one could expect the larger model(ResNet50) performs better than ResNet18. Since the bigger model has more parameters and layers, it can learn higher number of patterns in the data which help it in qualifying the images, it can also learn patterns which are finer than what the ResNet18 is able to learn. The trade-off which has to be considered is the size of the models. ResNet50 is slightly more than double the size of ResNet18, meaning it would have to be pruned an extremely significant percentage. Pruning a model past 40% is starting to get extreme in a task as such, since we are already working with minimal data and the resulting accuracies arent too great when it comes to pruning. We also have to consider the fact that the model is supposed to be as small as possible. We wish to make the model as small as possible such that we can integrate it into the embedded system of PillCam, which ofcourse has minimal memory. Thus, taking these factors into consideration and trying to minimize the model and maximize the accuracy, it is not wise to use ResNet50 since it is so much large than ResNet18.

Using Cross-Validation in this task was an important feature, but it came with a small initially unforeseen problem. This problem was *Data Leakage*. This simply means that some part of the testset data was used in the training and validation parts. This means that the model performed better on the testset than it should, leading to corrupted testing. This problem was solved by splitting the data in a seeded manner such that after restarting the Jupiter Notebook Kernel, the data would not be randomly split again into different sets, thus stabilizing the data used for training and testing.

When training these type of models, computational power is something of concern, as it might crash if the memory is overflowed with too big of a batch in training. Thus, the training was done using 5fold cross-validation. With the augmented data the models took some time training, and while using CUDA i had to be carefull considering

the memory space in the graphic cards used. The graphic card used in these experiments was RTX 3060 with 12Gb memory, which helped train the models quicker than simply overloading the CPU.

Another important thing to mention is that the data in quality task were labeled by medical professionals which made the data noisy, thus, making it worse quality for training. This then affects the performance of the models, which was one of the challenges in this task. See below for the different image classes.



Fig. 9: Random image from the quality dataset of class 0



Fig. 10: Random image from the quality dataset of class 1



Fig. 11: Random image from the quality dataset of class 2



Fig. 12: Random image from the quality dataset of class 3

1) *Too aggressive Pruning:* Training the ResNet50 model from default weights was not possible using the process which was coded and used for pruning of ResNet18. This was due to the aggressive pruning implemented. The model could not learn fast enough as it was pruned too much for each epoch. Thus, i realized it needed to be fully pre-trained before pruning, since it was much larger than ResNet18 and needed more training before it could be pruned effectively. Since i mention this problem, i should also say that the ResNet18 model was trained with no problems using the method which did not work for ResNet50. To solve the problem i simply loaded the best ResNet50 model weights from training done earlier. This pretrained model used the images which were rotated 45degrees, 90degrees etc., meaning the full dataset, and had accuracy on the testset at 96.25%. Making this model the best in the batch of 5 folds. The process of pruning was

slightly changed as well. The Pruning was done once at the beginning of each epoch, meaning the pruning happened before the batch forward pass. This was slightly less aggressive, as in the original pruning method, pruning was done before the forwardpass and backward pass. This same method could have been used in pruning the ResNet50 as it is training, but better and more hardware would be necessary to speed up the training.

After applying correct pruning techniques we can view in figures 7 and 8. The ResNet18 model can only be pruned to 30 percent before dipping under 70 percent, while ResNet50 can be pruned for percentages above 90% before falling below 70% accuracy on the testset. This fact tells us that ResNet50 holds extremely many parameters which are of very low importance.

But, even after pruning, the computational graph of ResNet50 may remain more complex than ResNet18. Without specialized libraries or hardware to leverage sparsity, the inference time might not improve proportionally. Although, considering the task, we do need to learn many details in the images, which ResNet50 learns better and more of than ResNet18. Thus, further considerations should be applied and more detailed goal of the task is needed to make any decisions in which model is more applicable.

VI. CONCLUSION

A. Wall or Rest Task

In the Wall or Rest classification task, our primary goal was to develop a model capable of efficiently distinguishing between diagnostically valuable images and non-informative 'wall' images captured by the PillCam. The ResNet18 architecture was an optimal choice for this binary classification problem due to its balance of simplicity, performance, and efficiency.

ResNet18's relatively shallow depth and lightweight make it computationally less demanding, which is advantageous for deployment on embedded systems with limited resources, such as the PillCam. The model achieved high accuracy in differentiating between the two classes.

An important aspect of the findings is ResNet18's robustness to pruning. it was possible to prune the model up to 40% of the model's weights without a significant drop in classification accuracy, maintaining above 90% accuracy on the test set. This level of sparsity reduces the model's size and computational requirements.

In practical terms, deploying a pruned ResNet18 model within the PillCam system can lead to several benefits:

- **Reduced Inference Time:** With fewer parameters to process, the model can perform faster predictions, which is essential for real-time image analysis.

- **Lower Energy Consumption:** A smaller, more efficient model requires less computational power, conserving the PillCam’s limited energy resources and potentially extending its operational lifespan.

In conclusion, the ResNet18 model, particularly when pruned, offers a great solution for the Wall or Rest classification task.

B. Quality Task

In the quality classification task, we observed that ResNet50, despite its larger size compared to ResNet18, achieved higher accuracy both before and after pruning. The additional layers and parameters in ResNet50 allowed it to capture more complex features and subtle distinctions necessary for classifying image quality, which was particularly important given the limited and noisy dataset. It kept acceptable performance even when pruned up to 90%, whereas ResNet18’s accuracy dropped significantly when pruned beyond 10%.

This robustness indicates that ResNet50 contains a substantial number of weak parameters that can be pruned without severely affecting its performance. The ability to prune ResNet50 while retaining high accuracy makes it a promising candidate for applications where high performance and reduced model size is needed.

However, the pruned ResNet50, even when having fewer active parameters, still retains a complex architecture that may not be ideal for deployment on resource-constrained devices like the PillCam. The sparsity introduced through unstructured pruning does not inherently reduce the model’s computational complexity or inference time unless specialized hardware or software optimizations are employed to leverage the sparse structure.

In conclusion, while ResNet50 offers better performance and greater pruning potential for the quality classification task, practical deployment considerations must be considered. Future work should focus on exploring structured pruning methods, quantization, or alternative efficient architectures like MobileNetV2 or EfficientNet, which may offer a better balance between performance and model complexity.

REFERENCES

- [1] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” 2017. [Online]. Available: <https://arxiv.org/abs/1412.6980>
- [2] M. A. Raihan and T. M. Aamodt, “Sparse weight activation training,” 2020. [Online]. Available: <https://arxiv.org/abs/2001.01969>
- [3] T. Hoefler, D. Alistarh, T. Ben-Nun, N. Dryden, and A. Peste, “Sparsity in deep learning: Pruning and growth for efficient inference and training in neural networks,” 2021. [Online]. Available: <https://arxiv.org/abs/2102.00554>